Efficient Data Structures for
RDD-based Apriori on Spark

Luis Alberto Vásquez Vargas
IC – UNICAMP

MO809 – Topics in Distributed Computing
Instructor: Prof. Luiz Fernando Bittencourt

l322989@dac.unicamp.br
April 7, 2026

Outline

1. The Base Study
2. Introduction & Motivation
3. The FIM Problem
4. Infrastructure Evolution
5. Literature Overview
6. Proposed Data Structures
7. Experimental Results
8. Conclusion
9. References

The Base Study

Original Research Paper

A Data Structure Perspective to the RDD-based Apriori Algorithm on Spark

- **Source:** *International Journal of Information Technology* (Springer), 2019.
- **Keywords:** Parallel Algorithms, Big Data, Apriori, Spark, RDD.
- **Authors & Affiliations:**
  - **Pankaj Singh & P. K. Mishra:** Dept. of Computer Science, BHU.
  - **Sudhakar Singh:** Dept. of Electronics & Comm., Univ. of Allahabad.
  - **Rakhi Garg:** Mahila Maha Vidyalaya, BHU, Varanasi.

Pankaj Singh

Sudhakar Singh

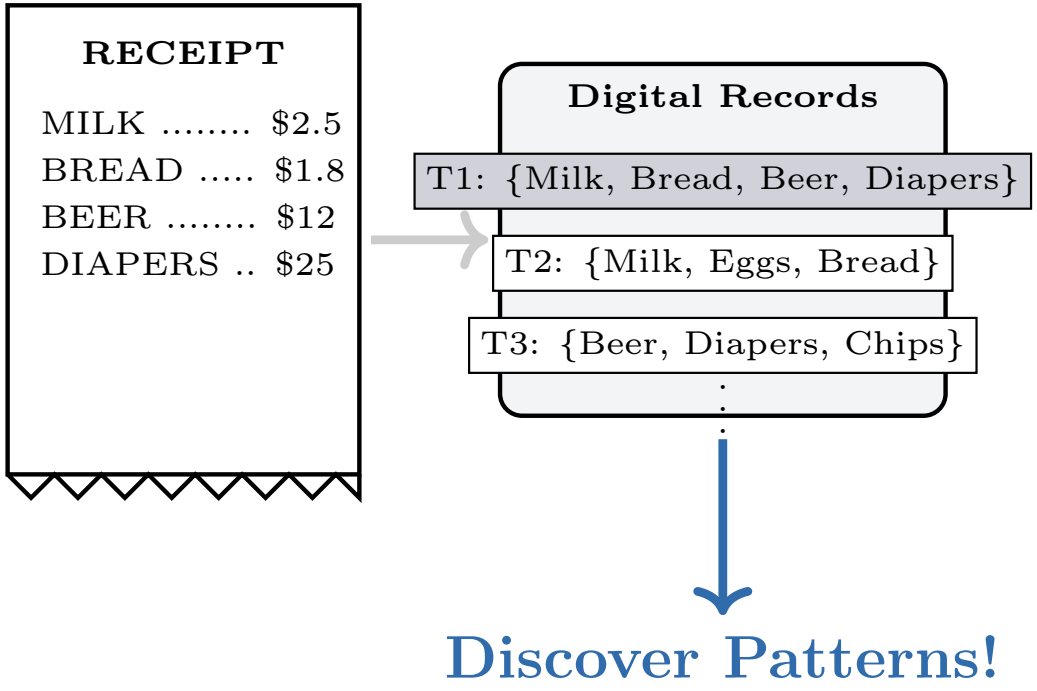
P. K. Mishra

Rakhi Garg

Focus: Comparative efficiency of **Hash Tree**, **Trie**, and **Hash Table Trie** in distributed Spark environments.

Introduction: The Market Basket Problem

- **The Scenario:** A retail store processes millions of transactions daily.
- **The Question:**
  - Which items are frequently bought together?
  - $\{Milk, Bread\} \rightarrow \{Eggs\}$?
  - $\{Beer\} \rightarrow \{Diapers\}$?
- **The Value:**
  - Cross-selling & Promotions.
  - Inventory management & Layout.
  - Catalog optimization.



The FIM Problem

- **Frequent Itemset Mining (FIM):** Finding subsets with frequency $\geq$ *minsup*.
- **The "Bundle Deal" Analogy:**
  - If customers buy GPUs, what else do they buy?
  - $n = 1000$ items lead to 2^{1000} possible combinations.

- **The Apriori Principle:**
  "Any subset of a frequent itemset must also be frequent."
- We prune the space, but we need efficient storage for millions of candidates.

Infrastructure Evolution: Apache Spark

- **The Paradigm Shift:** Moving from MapReduce (Disk-based) to **Spark (Memory-based)**.
- **RDD (Resilient Distributed Dataset):**
  - Immutable, partitioned collections of records.
  - Distributed across the RAM of multiple cluster nodes.

- **The Challenge for Apriori:**
  - Apriori requires multiple passes over the dataset.
  - Spark keeps the dataset in RAM, but the **CPU overhead** of searching candidate itemsets in every transaction remains the primary bottleneck.

Infrastructure Evolution: Spark RDD Workflow

- **Parallelism:** Each worker node processes a partition of the transaction RDD.
- **Candidate Generation:**
  - Drivers broadcast the candidate itemsets to all workers.
  - Workers must match these candidates against local transactions.

- **The Need for Speed:**
  - If the data structure used for matching is slow, the distributed RAM advantage is neutralized by **search latency**.

Literature Overview: FIM Evolution

- **The Classic Era (Sequential):**
  - **Apriori (1994):** The foundation. Used Hash Trees to store candidates.
  - **FP-Growth (2000):** Eliminated candidate generation using a prefix-tree (FP-Tree), but is notoriously difficult to parallelize efficiently.
- **The Distributed Era (Disk-based):**
  - **PFP (2008):** Parallel FP-Growth on MapReduce. Heavy Disk I/O overhead due to the nature of Hadoop.
- **The Modern Era (In-Memory):**
  - **Spark-based Apriori:** Focuses on RDD persistence to avoid re-reading data.
  - **The Gap:** Most Spark implementations still rely on standard Hash Trees or simple Arrays, failing to optimize the CPU-to-Memory bottleneck.

Literature Overview: Data Structure Benchmarks

- **Hash Tree (HT):**
  - Standard for decades.
  - *Issue:* High branch misprediction and poor cache locality in modern CPUs.

- **Trie (Prefix-Tree):**
  - Fast traversal, no hashing overhead at each node.
  - *Issue:* Can become very deep and sparse, consuming significant memory.

- **The Research Question:**
  *Can a hybrid approach—the **Hash Table Trie**—outperform both in a distributed RDD environment where memory is abundant but CPU cycles are precious?*

Data Structures: The Hash Tree

- **Structure:** Internal nodes contain hash tables; leaves store candidate itemsets.
- **Traversal Logic:**
  1. Hash the i-th item of a transaction.
  2. Follow the corresponding branch.
  3. Repeat until a leaf is reached and verify the candidate.

- **The Spark Bottleneck:**
  - **High Collision:** Multiple itemsets mapping to the same leaf force slow linear searches.
  - **CPU Overhead:** Repeated hashing at every level is expensive when scaled across RDD partitions.

Data Structures: The Trie (Prefix Tree)

- **Structure:** A rooted tree where each node represents a single item.
- **Mechanism:**
  - Paths from root to leaf represent full itemsets.
  - Common prefixes are shared, reducing redundant storage.

- **Advantage in RDDs:**
  - **Zero Hashing:** Navigates purely by pointers/references.
  - **Predictable Traversal:** If a prefix isn't in the transaction, the entire branch is pruned instantly.
  - *Trade-off:* Searching for a child node among many siblings can become $O(k)$ where k is the alphabet size.

Data Structures: The Hash Table Trie

- **The Hybrid Concept:**
  - A Trie where each node uses a **local Hash Table** to store its children.
- **Why it dominates in Spark:**
  - **Constant Time ($O(1)$):** Finding the next item in a sequence doesn't require scanning a list of siblings.
  - **CPU Efficiency:** It combines the pruning power of a Trie with the lookup speed of Hashing.
  - **Memory-Speed Tradeoff:** Uses more RAM than a standard Trie, but in an RDD environment, RAM is exactly what we have to spare.

The Verdict

The Hash Table Trie fixes the "branching bottleneck" of standard Tries while maintaining better cache locality than traditional Hash Trees.

Experimental Results: Benchmark Setup

- **Cluster Environment:**
  - Apache Spark 2.x cluster.
  - RDDs persisted in-memory (Storage Level: MEMORY_ONLY).
- **Datasets:**
  - **Retail:** Sparse, long transactions.
  - **T10I4D100K:** Standard synthetic benchmark for Apriori.
  - **Mushroom/Chess:** Dense datasets with high candidate overlap.
- **Metric:** Execution time (seconds) per iteration (L_k generation).

Experimental Results: Performance Analysis

- **Observation 1: Hash Tree Bottleneck**
  - Consistently slower due to hash collisions and leaf-node linear searches.
  - Performance degrades sharply as *minsup* decreases (more candidates).
- **Observation 2: Trie Efficiency**
  - Significant speedup over Hash Trees due to prefix pruning.
  - CPU usage is lower, but still limited by sibling-node traversal in dense layers.
- **Observation 3: Hash Table Trie (HTT) Dominance**
  - **Peak Performance:** HTT shows the lowest execution time across all datasets.
  - **Scalability:** As candidate counts reach millions, HTT maintains $O(1)$ child lookup, outperforming the standard Trie by 15–20%.

Experimental Results: Summary Table

Relative Speedup vs. Hash Tree (HT)

Dataset	Hash Tree	Trie	HT Trie
Retail	1.0x (Baseline)	1.4x	**1.8x**
T10I4D100K	1.0x (Baseline)	2.1x	**2.5x**
Mushroom	1.0x (Baseline)	1.9x	**2.3x**

The RDD Conclusion

The results validate that within Spark, the computational efficiency of $O(1)$ lookups in the Hash Table Trie justifies its higher memory footprint.

Conclusion: The Sequential Context

- **Theoretical Expectation:**
  - The **Hash Table Trie** was designed to accelerate itemset traversal by combining Trie logic with hashing.

- **Previous Practical Failure:**
  - In earlier sequential implementations, where **memory was a strict bottleneck**, the hashing overhead caused it to underperform.
  - It was often documented as the least efficient variant in non-distributed settings.

Conclusion: The Spark Paradigm Shift

- **Distributed Advantage:**
  - Within Spark's **distributed memory (RDD)** architecture, the Hash Table Trie finally realizes its theoretical efficiency.

- **Benchmark Results:**
  - Experimental data shows it performs **significantly better** than the traditional Hash Tree.
  - It maintains a slight edge over the standard Trie across most datasets.

Conclusion: Final Verdict

- **In-Memory Optimization:**
  - Memory-optimized structures are the key to high-performance FIM on Spark.

- **Summary:**
  - The **Hash Table Trie**, once considered the "worst" in sequential settings, is now the most robust choice for RDD-based Apriori.
  - Infrastructure alone (Spark) isn't enough; the underlying **data structure** must evolve to match the hardware.

References I

P. Singh, S. Singh, P. K. Mishra, and R. Garg.
A data structure perspective to the RDD-based Apriori algorithm on Spark.
Int. J. Inf. Technol., 14(3):1585–1594, 2019.

M. J. Zaki.
Scalable algorithms for association mining.
IEEE Trans. Knowl. Data Eng., 12(3):372–390, 2000.

C. Borgelt.
Frequent item set mining.
WIREs Data Mining Knowl Discov, 2:437–456, 2012.

B. Goethals.
Survey on Frequent Pattern Mining.
Helsinki Institute for Information Technology (HIIT), 2003.

References II

C. Borgelt.
Efficient Implementations of Apriori and Eclat.
Workshop on Frequent Itemset Mining Implementations (FIMI), 2003.

R. Agrawal and R. Srikant.
Fast algorithms for mining association rules.
Proc. 20th Int. Conf. VLDB, 487–499, 1994.

H. B. Abdalla, Y. Kumar, Y. Zhao, and D. Tosi.
A Comprehensive Survey of MapReduce Models for Processing Big Data.
Algorithms, 13(11), 2020.

C. C. Aggarwal.
Data Mining: The Textbook.
Springer, 2015.

Questions?

Luis Alberto Vásquez Vargas
IC – UNICAMP

l322989@dac.unicamp.br